Recursion with Array

1. Understanding Recursion with Array

What is Recursion?

Recursion is a programming technique where a method calls itself to solve a problem. In recursion, we break down a complex problem into smaller, similar subproblems.

Key Components of Recursion

1. **Base Case**: The condition that stops the recursion
2. **Recursive Case**: The method calling itself with modified parameters

Basic Structure of Recursive Method

```java
public returnType recursiveMethod(parameters) {
    // Base case
    if (condition) {
        return baseValue;
    }

    // Recursive case
    return recursiveMethod(modifiedParameters);
}
```

Working with Arrays in Recursion

Example 1: Finding Sum of Array Elements

```java
public static int sumArray(int[] arr, int index) {
    // Base case: if index reaches array length
    if (index >= arr.length) {
        return 0;
    }

    // Recursive case: current element + sum of remaining elements
    return arr[index] + sumArray(arr, index + 1);
}

// Usage
public static void main(String[] args) {
    int[] numbers = {1, 2, 3, 4, 5};
    int total = sumArray(numbers, 0);
    System.out.println("Sum: " + total); // Output: Sum: 15
}
```

Example 2: Finding Maximum Element in Array

```java
public static int findMax(int[] arr, int index) {
    // Base case: if only one element left
    if (index == arr.length - 1) {
        return arr[index];
    }

    // Recursive case: compare current element with max of remaining
    int maxOfRest = findMax(arr, index + 1);
    return Math.max(arr[index], maxOfRest);
}

// Usage
public static void main(String[] args) {
    int[] numbers = {3, 7, 2, 9, 1};
    int maximum = findMax(numbers, 0);
    System.out.println("Maximum: " + maximum); // Output: Maximum: 9
}
```

Example 3: Printing Array Elements

```java
public static void printArray(int[] arr, int index) {
    // Base case: if index reaches array length
    if (index >= arr.length) {
        return;
    }

    // Print current element
    System.out.print(arr[index] + " ");

    // Recursive case: print remaining elements
    printArray(arr, index + 1);
}

// Usage
public static void main(String[] args) {
    int[] numbers = {10, 20, 30, 40};
    printArray(numbers, 0); // Output: 10 20 30 40
}
```

Example 4: Searching Element in Array

```java
public static boolean searchElement(int[] arr, int target, int index) {
    // Base case: if index reaches array length
    if (index >= arr.length) {
        return false;
    }

    // Base case: if element found
    if (arr[index] == target) {
        return true;
    }

    // Recursive case: search in remaining array
    return searchElement(arr, target, index + 1);
}

// Usage
public static void main(String[] args) {
    int[] numbers = {5, 2, 8, 1, 9};
    boolean found = searchElement(numbers, 8, 0);
    System.out.println("Found: " + found); // Output: Found: true
}
```

Example 5: Reverse Array Using Recursion

```java
public static void reverseArray(int[] arr, int start, int end) {
    // Base case: if start >= end
    if (start >= end) {
        return;
    }

    // Swap elements at start and end positions
    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;

    // Recursive case: reverse remaining array
    reverseArray(arr, start + 1, end - 1);
}

// Usage
public static void main(String[] args) {
    int[] numbers = {1, 2, 3, 4, 5};
    reverseArray(numbers, 0, numbers.length - 1);

    // Print reversed array
    for (int num : numbers) {
        System.out.print(num + " "); // Output: 5 4 3 2 1
    }
}
```

Important Tips for Array Recursion

1. Always define a clear base case to prevent infinite recursion
2. Make sure the recursive call moves toward the base case
3. Use index parameters to track position in the array
4. Consider the direction of traversal (forward or backward)
5. Be mindful of array bounds to avoid IndexOutOfBoundsException

2. Practice Questions

Question 1: Count Even Numbers

Write a recursive method to count the number of even numbers in an array.

```java
public static int countEven(int[] arr, int index) {
    // Your implementation here
}
```

Question 2: Find Minimum Element

Write a recursive method to find the minimum element in an array.

```java
public static int findMin(int[] arr, int index) {
    // Your implementation here
}
```

Question 3: Calculate Product of Array Elements

Write a recursive method to calculate the product of all elements in an array.

```java
public static int productArray(int[] arr, int index) {
    // Your implementation here
}
```

Question 4: Check if Array is Sorted

Write a recursive method to check if an array is sorted in ascending order.

```java
public static boolean isSorted(int[] arr, int index) {
    // Your implementation here
}
```

Question 5: Binary Search Using Recursion

Write a recursive method to perform binary search on a sorted array.

```java
public static int binarySearch(int[] arr, int target, int left, int right) {
    // Your implementation here
}
```

Quiz Questions

Question 1: What is the most important component of a recursive method? a) Loop structure b) Base case c) Array declaration d) Return statement

Answer: b) Base case

Question 2: In the following recursive method, what happens if the base case is missing?

```java
public static int sum(int[] arr, int index) {
    return arr[index] + sum(arr, index + 1);
}
```

a) The method returns 0
b) Compilation error occurs
c) Stack overflow occurs
d) The method returns the first element

Answer: c) Stack overflow occurs

Question 3: What will be the output of this recursive method call?

```java
public static void print(int[] arr, int index) {
    if (index >= arr.length) return;
    System.out.print(arr[index] + " ");
    print(arr, index + 1);
}
// Called with: print({1, 2, 3}, 0)
```

a) 3 2 1
b) 1 2 3
c) 1 1 1
d) No output

Answer: b) 1 2 3

Question 4: Which parameter is commonly used to track position in array recursion? a) length b) size c) index d) counter

Answer: c) index

Question 5: In recursive array traversal, what typically happens to the index parameter in each recursive call? a) It stays the same b) It decreases by 1 c) It increases by 1 d) It becomes 0

Answer: c) It increases by 1

Practice Coding Questions

Question 1: Write a recursive method to find the second largest element in an array.

Question 2: Create a recursive method that returns the sum of elements at even indices in an array.

Question 3: Write a recursive method to check if a given array is a palindrome (reads the same forwards and backwards).

Question 4: Implement a recursive method to merge two sorted arrays into one sorted array.

Question 5: Write a recursive method to find the number of occurrences of a specific element in an array.